Deep Learning

CMU PhD Workshop on Computational Economics

John Stachurski

April 2026

Topics

- What are artificial neural networks?
- How are they trained on data?
- Why is DL so successful?
- Rolling our own: DL with JAX

Artificial neural networks (ANNs) are the core of deep learning

DL = training ANNs with multiple hidden layers

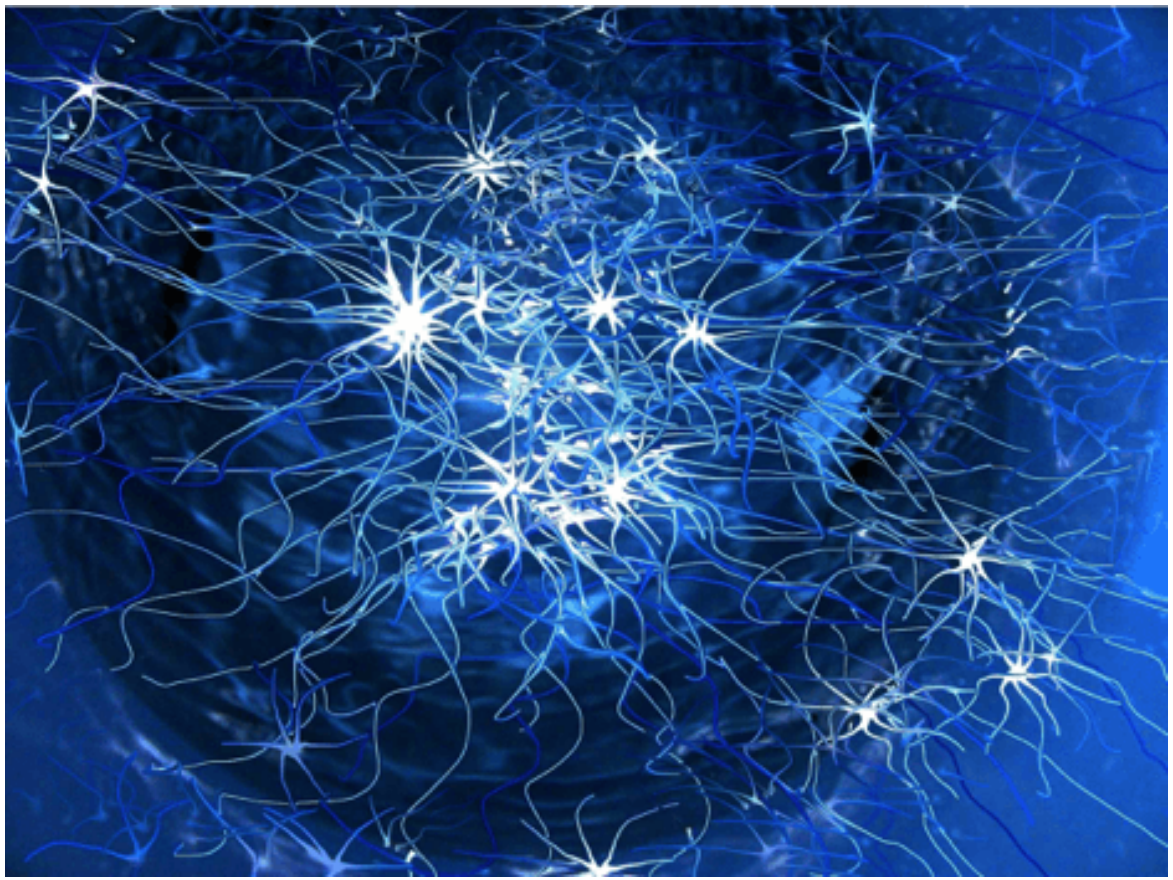
Major successes:

- Natural language processing
- Image recognition
- Speech recognition
- Games
- LLMs

Brief history

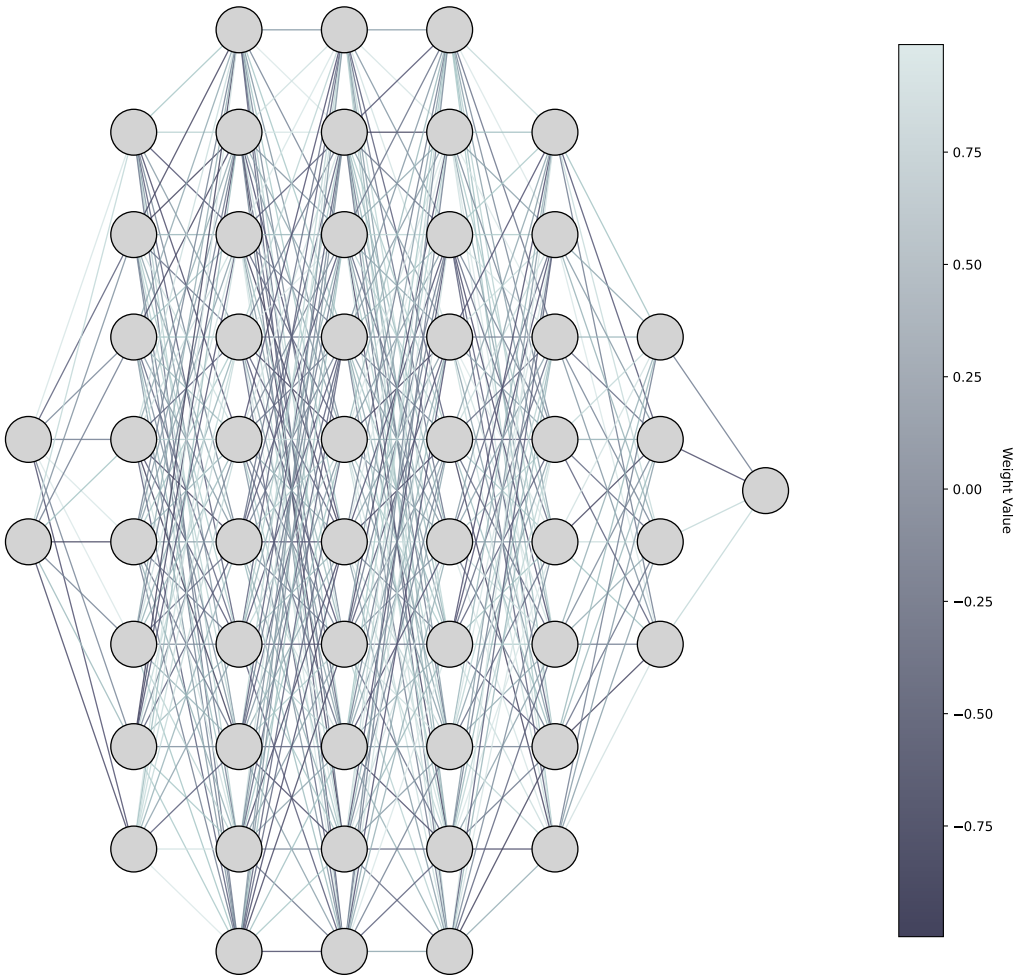
- 1940s–1980s: mathematical models of NNs, perceptrons, backpropagation
- 1990s–2000s: SVMs rival ANNs; DL starts finding successes in large problems
- Since ~2012: explosion of progress — CNNs, RNNs, transformers, LLMs, etc.

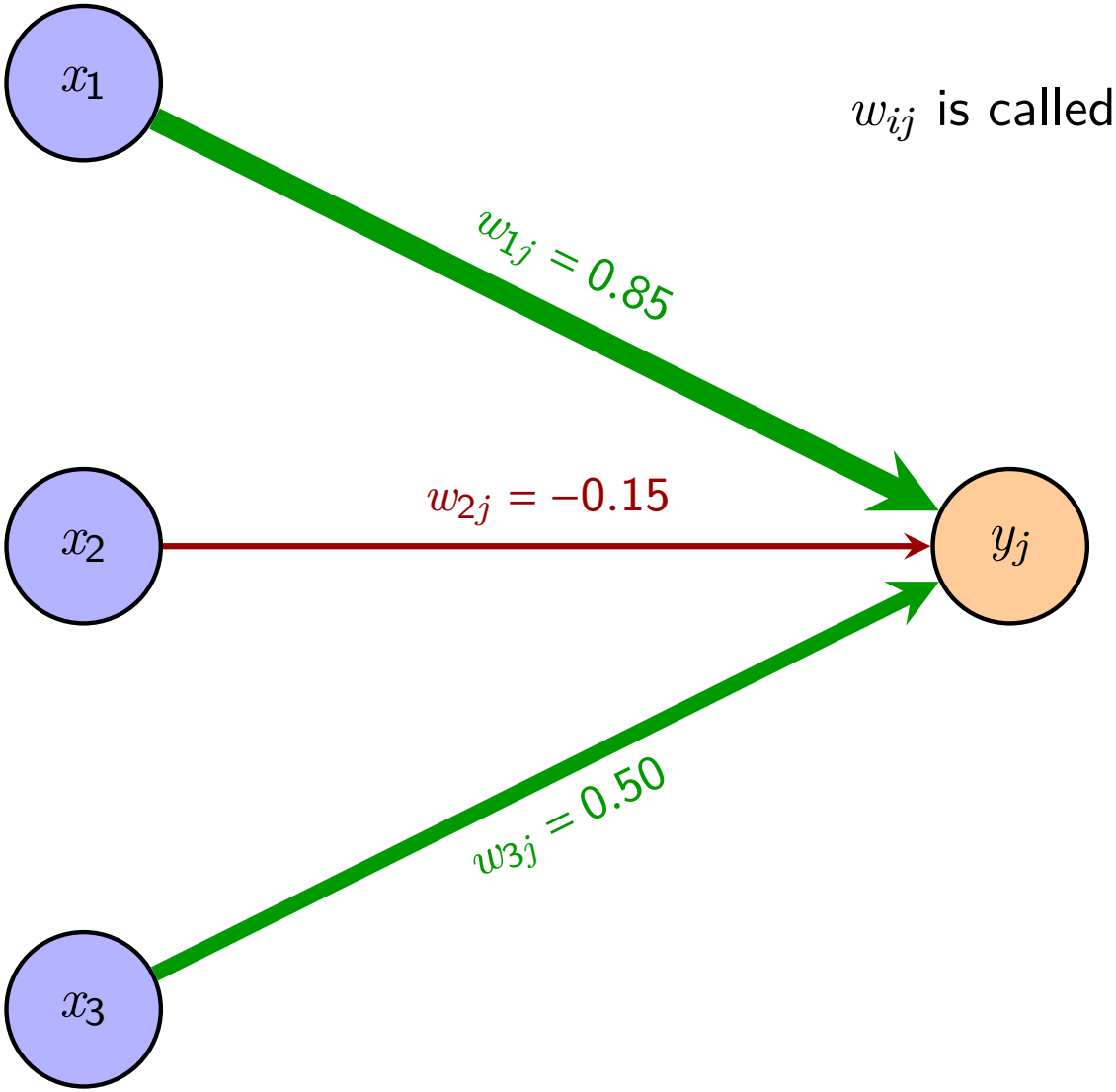
A model of the human brain



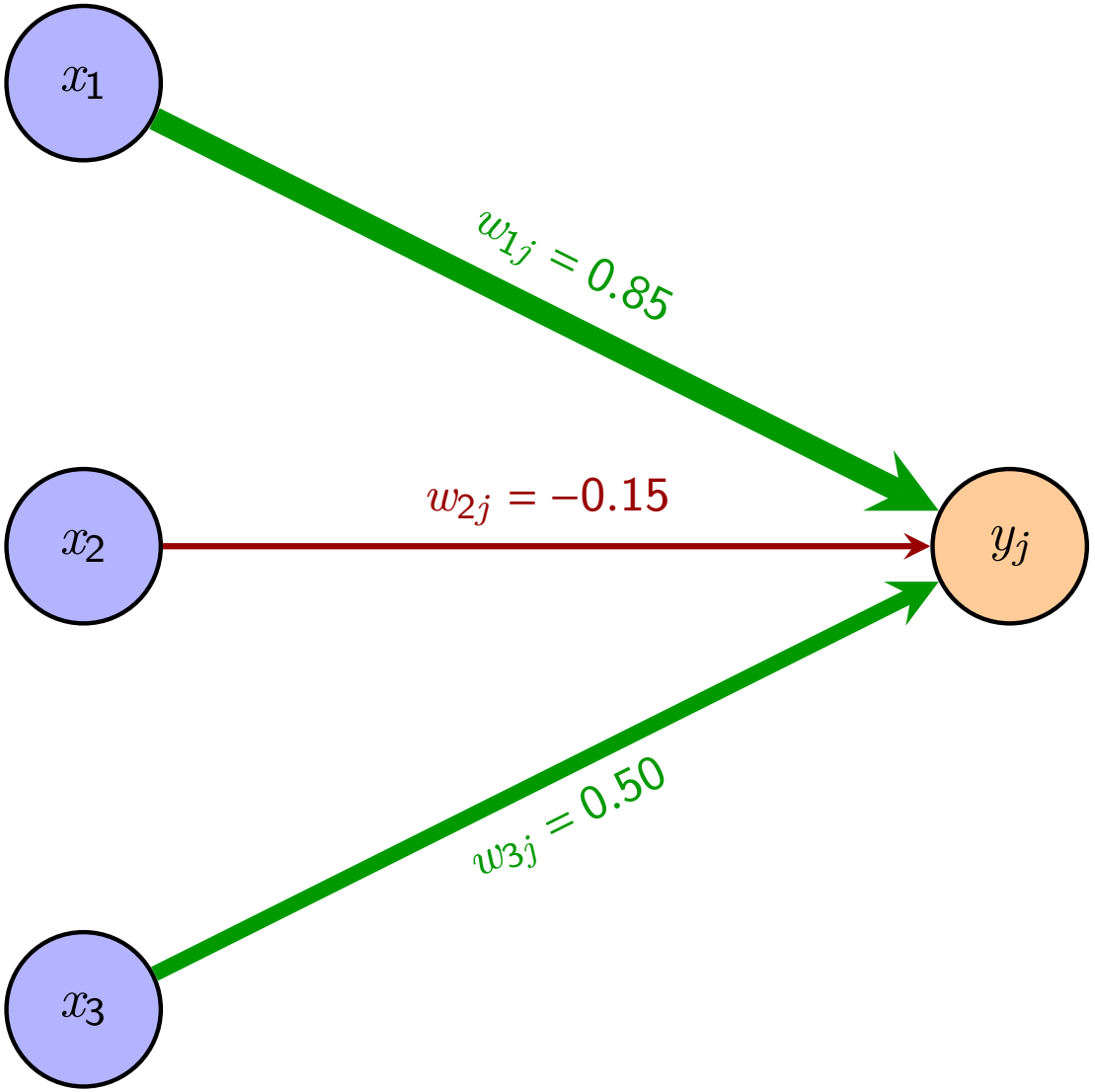
– source: Dartmouth undergraduate journal of science

A mathematical representation: directed acyclic graph

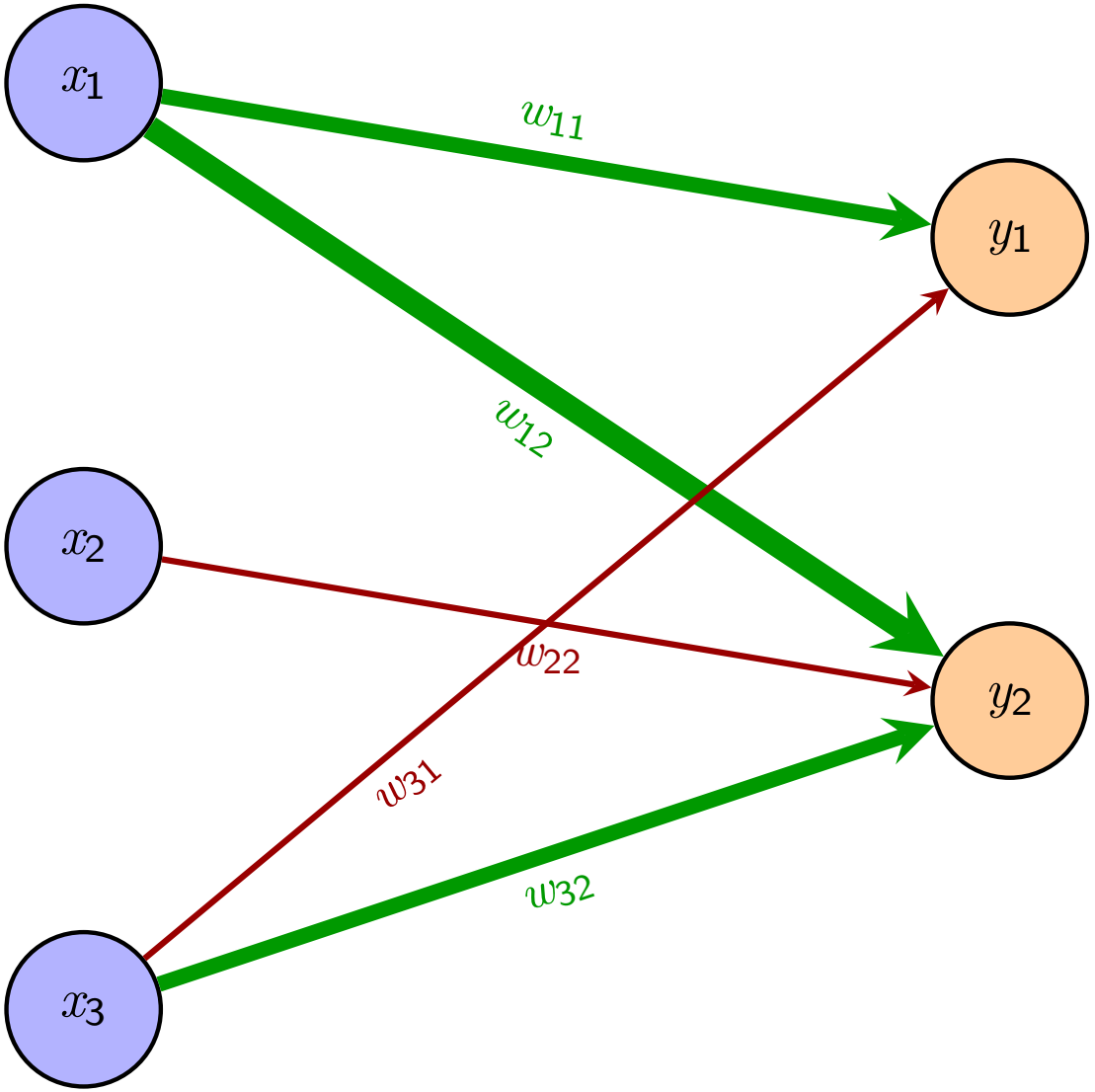




w_{ij} is called a "weight"



$$y_j = \sum_i x_i w_{ij}$$



$$y_1 = \sum_i x_i w_{i1}$$

$$y_2 = \sum_i x_i w_{i2}$$

$$\implies y = xW$$

Note that we are using row vectors: $y = xW$

This is natural given our notation

- w_{ij} points from i to j
- hence $y_j = \sum_i x_i w_{ij}$ (total flow of activation to node j)
- hence $y = xW$

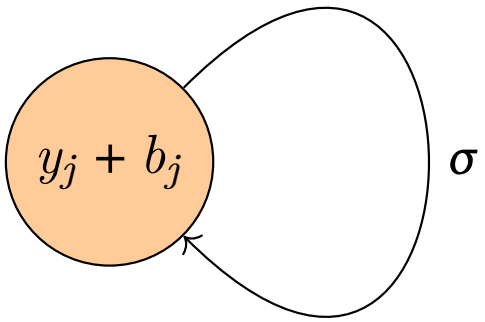
Also fine to use column vectors — just transpose W

Next steps

After computing $y_j = \sum_i x_i w_{ij}$ we

1. add a bias term b_j and
2. apply a nonlinear "activation function" $\sigma\colon \mathbb{R} \to \mathbb{R}$

applying activation function



First add bias:

$$y_j = \sum_i x_i w_{ij} \qquad \rightarrow \qquad y_j = \sum_i x_i w_{ij} + b_j$$

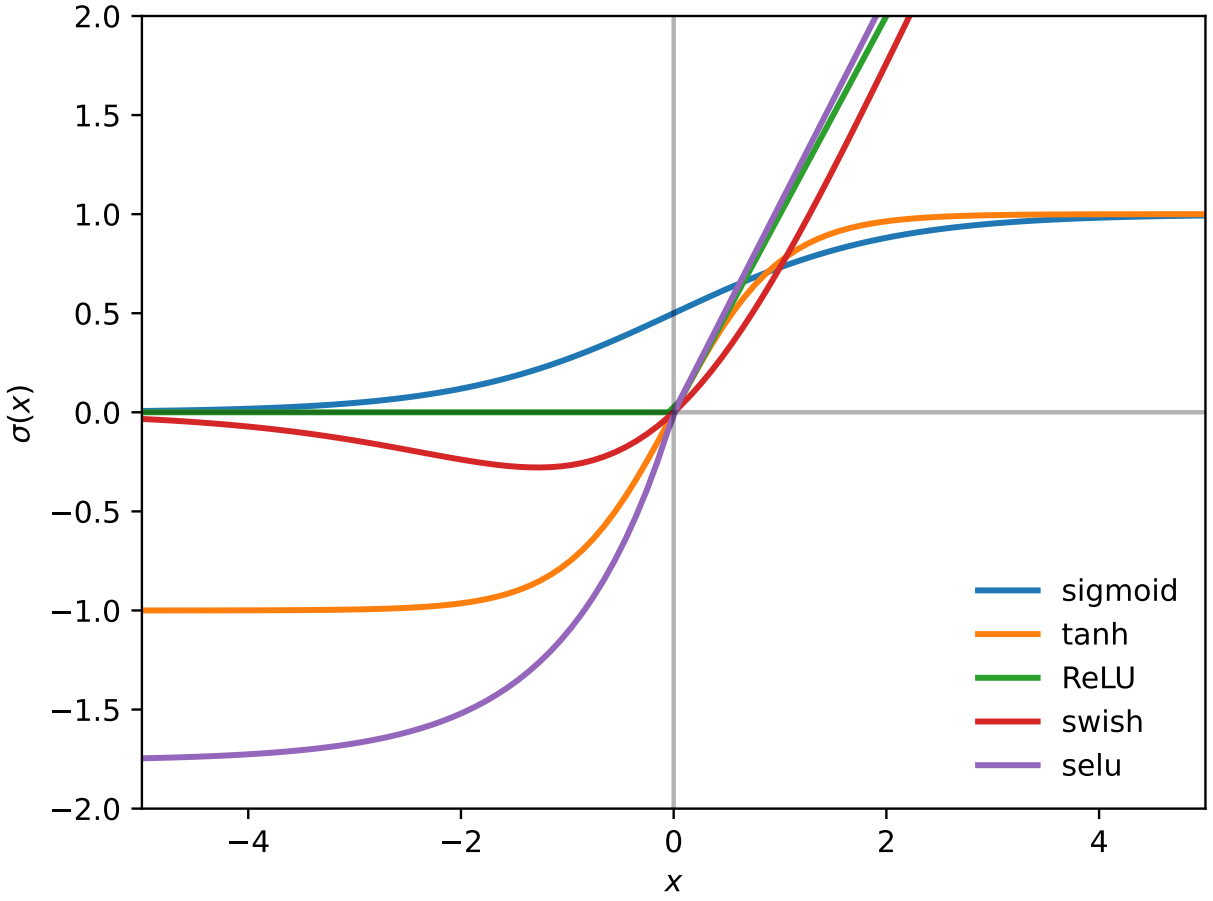
Then apply activation:

$$y_j = \sum_i x_i w_{ij} + b_j \qquad \rightarrow \qquad y_j = \sigma\left(\sum_i x_i w_{ij} + b_j\right)$$

Applying σ pointwise, we can write this in vector form as

$$y = \sigma(xW + b)$$

Common activation functions



Definition of an ANN

Let's now put this together

An ANN is a function f_θ from $\mathbb{R}^n$ to $\mathbb{R}^m$ having the form

$$f_\theta = G_m \circ G_{m-1} \circ \cdots \circ G_2 \circ G_1$$

where

- σ_ℓ is an activation function
- $\theta := \{W_1, b_1, W_2, b_2, \ldots, W_m, b_m\}$
- $G_\ell(x) = \sigma_\ell(xW_\ell + b_\ell)$

Universal function approximation I

Theorem. (Cybenko–Hornik 1989, 1991) Let

- $K \subset \mathbb{R}^n$ be compact,
- f be a continuous functions from K to $\mathbb{R}^m$, and
- σ be a continuous map from $\mathbb{R}$ to itself

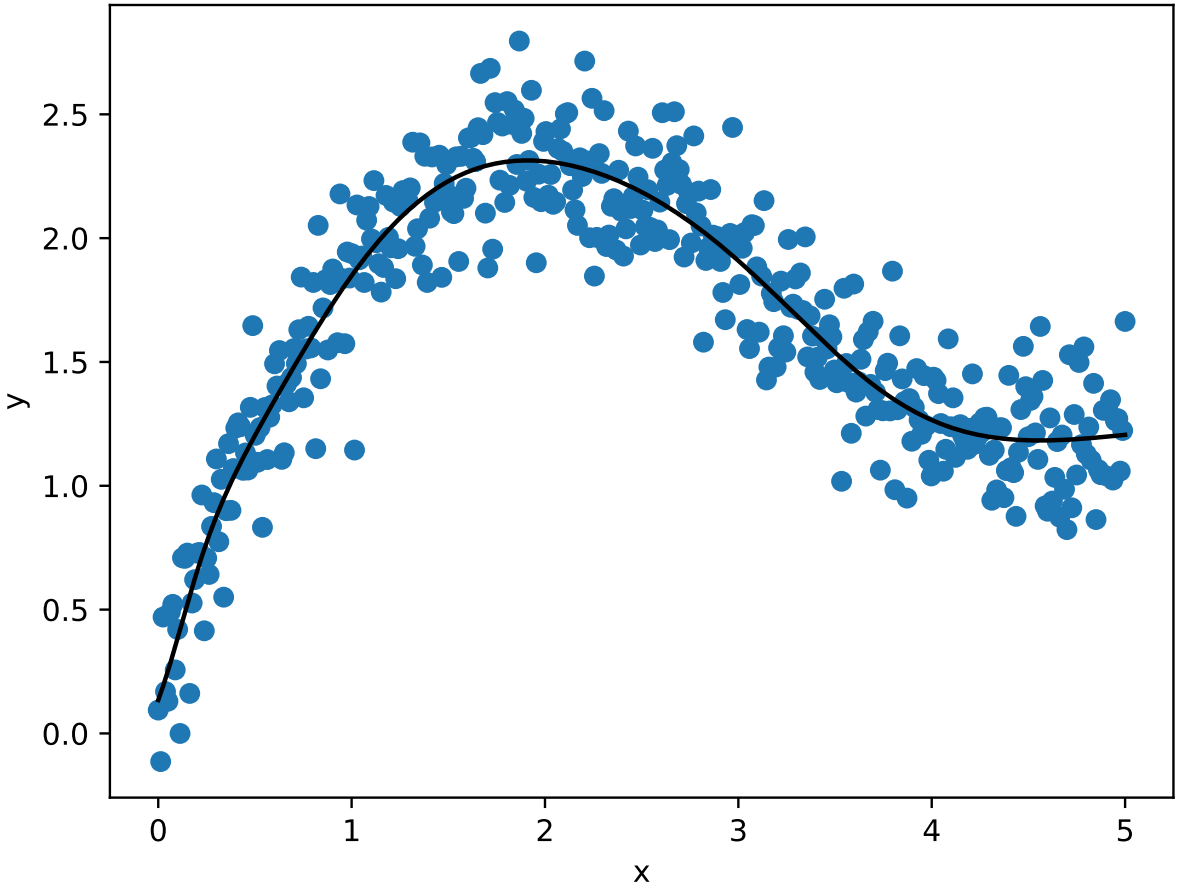
If σ is not polynomial, then, for every $\varepsilon > 0$, there exist an ANN $f_\theta \colon \mathbb{R}^n \to \mathbb{R}^m$ such that

$$\sup_{x \in K} \|f(x) - f_\theta(x)\| < \varepsilon$$

Before we get too excited: many function classes have the universal function approximation property

- polynomials in $C_0(K)$:= continuous functions on compacts
- linear span of an ONS in a Hilbert space
- SVMs with RBF kernels in reproducing kernel Hilbert space
- Gaussian processes with RBF kernels in $C_0(K)$
- random forests
- etc.

Training (Plain Vanilla)



An ANN is just a particular function f_θ

We want an ANN that is good at "prediction"

Steps

1. Observe data
2. Adjust the parameter vector θ to "fit" the data

Let's clarify these ideas

Suppose we wish to to predict output y from input x

- $x \in \mathbb{R}^j$
- $y \in \mathbb{R}^k$

Nonlinear regression: Choose parametric class $\{f_\theta\}_{\theta \in \Theta}$ and minimize

$$\ell(\theta) := \frac{1}{n} \sum_{i=1}^{n} \|y_i - f_\theta(x_i)\|^2$$

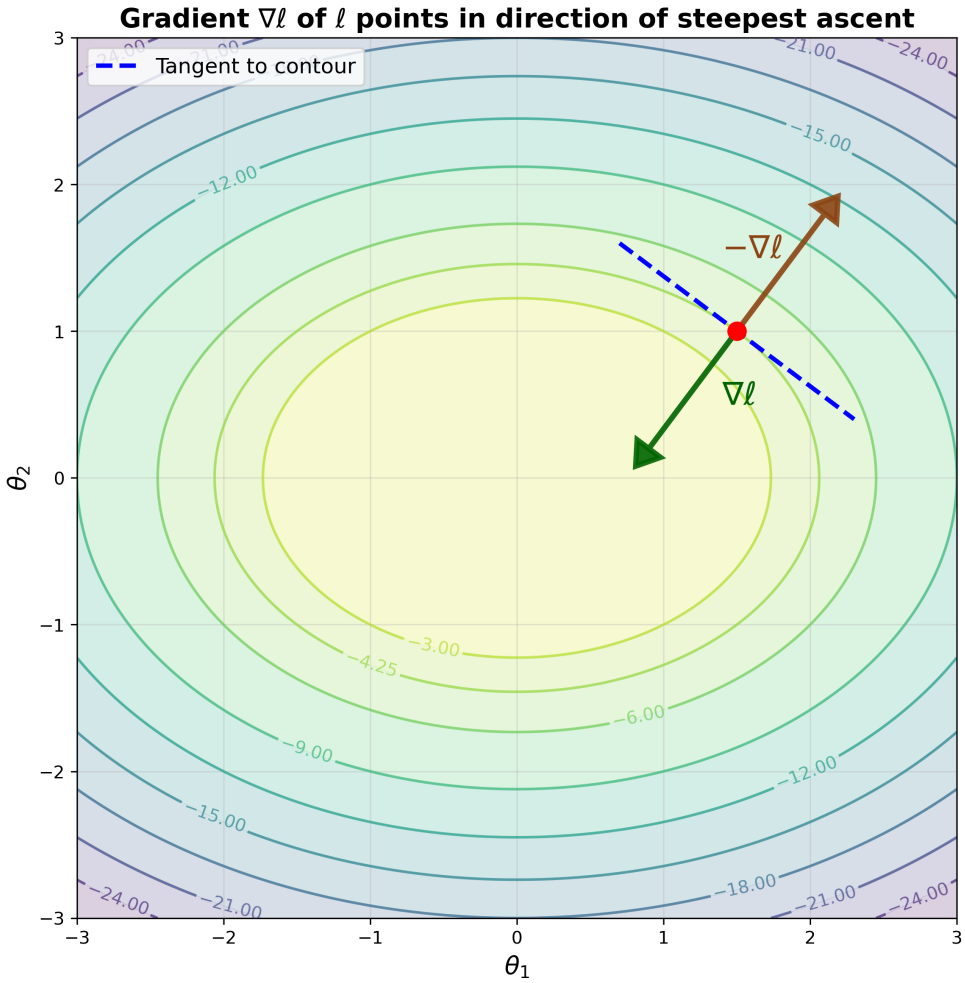
Deep learning: Nonlinear regression when $\{f_\theta\}$ is a class of ANNs:

$$f_\theta = G_m \circ G_{m-1} \circ \cdots \circ G_2 \circ G_1$$

where

- $G_\ell(x) = \sigma_\ell(xW_\ell + b_\ell)$
- $\theta := \{W_1, b_1, W_2, b_2, \ldots, W_m, b_m\}$
- σ_ℓ is an activation function

Minimizing the loss functions



Gradient descent

Algorithm: Implement $\nabla \ell$ and then update guess θ_k via

$$\theta_{k+1} = \theta_k - \lambda_k \cdot \nabla \ell(\theta_k)$$

- take a step in the opposite direction to the grad vector
- λ_k is the **learning rate**
- iterate until hit a stopping condition
- in practice replace $\ell(\theta)$ with batched loss $\rightarrow$ **SGD**

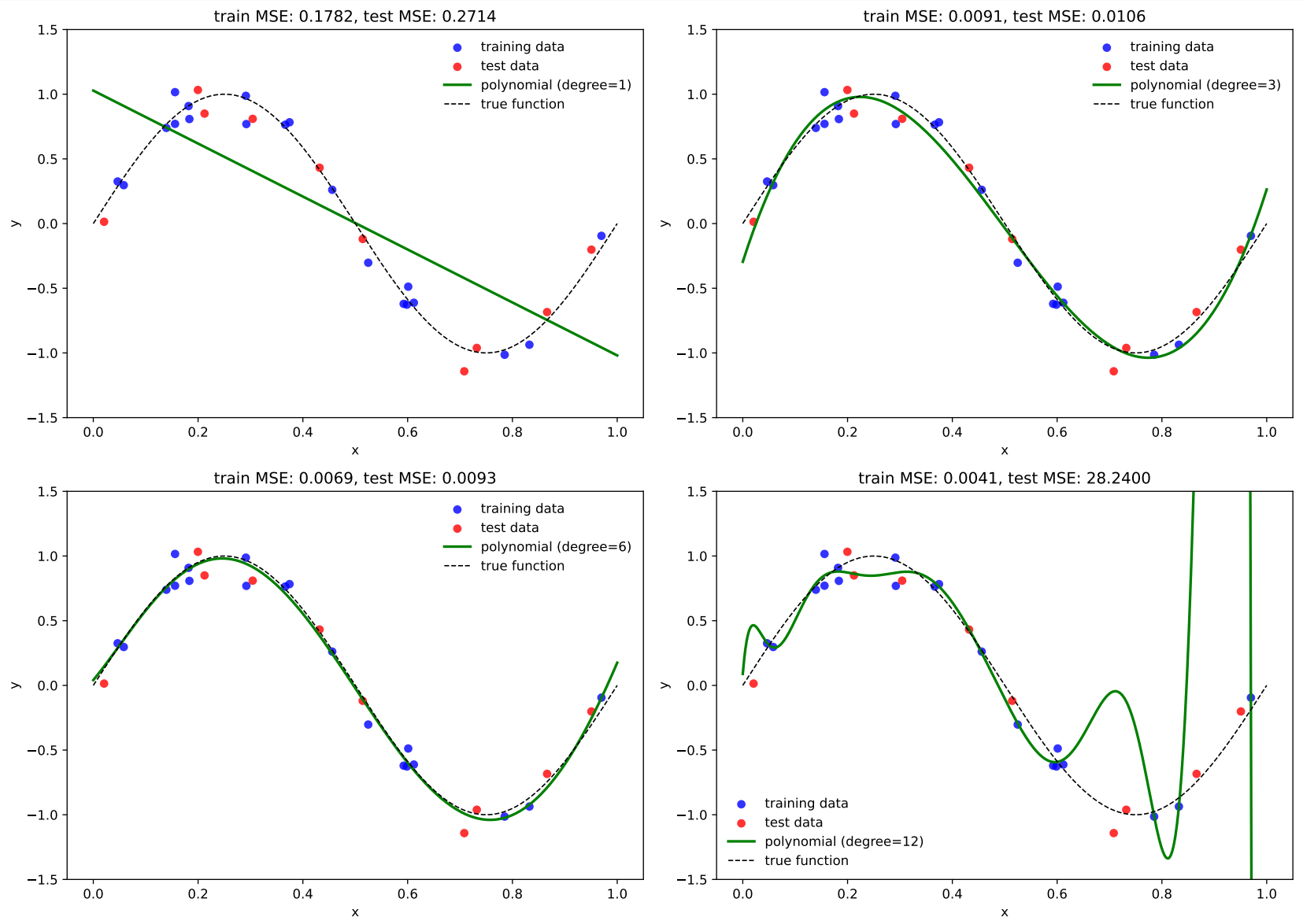
$$\frac{1}{|B|} \sum_{i \in B} \|y_i - f_\theta(x_i)\|^2$$

- many extensions: momentum, regularization, cross-entropy loss, etc.

Why do they work?

Why does DL work so well in so many cases??

Related question: what about overfitting?


train MSE: 0.1782, test MSE: 0.2714
training data
test data
polynomial (degree=1)
true function
train MSE: 0.0091, test MSE: 0.0106
training data
test data
polynomial (degree=3)
true function
train MSE: 0.0069, test MSE: 0.0093
training data
test data
polynomial (degree=6)
true function
train MSE: 0.0041, test MSE: 28.2400
training data
test data
polynomial (degree=12)
true function
x
y

If production-level DL models are so large, why don't they overfit?

Answer 1 Underlying relationships are complex – need complex model

Answer 2 Engineers avoid using full complexity of the model

- E.g., early stopping halts training when test loss starts to rise

Answer 3 Adding randomization prevents overfitting

- E.g., DropConnect and Stochastic Depth
- Stochastic gradient descent injects randomness

Randomness "adds some smoothing" to a given data set

"The model must learn to be robust to these perturbations, which encourages it to find smoother, more generalizable decision boundaries rather than fitting to noise in the training data."

Answer 4 Modern architectures have inductive biases built in

Egs.

- Translation invariance in CNNs
- Localization in CNNs – pixels influenced more by neighbor pixels
- Parameter sharing in RNNs – similarity of transformations across time

Injection of good priors guides learning

CS story

Because

- based on a model of the human brain!
- A universal function approximator!
- Can break the curse of dimensionality!

Really?

CS story

Because

- based on a model of the human brain!
- A universal function approximator!
- Can break the curse of dimensionality!

Really?

Claude AI

"Traditional methods typically require you to

- specify the intrinsic dimension,
- choose kernel parameters, or
- make structural assumptions about the manifold

Neural networks automatically adapt to intrinsic structure without requiring this prior knowledge"

Really?

Claude AI

"Traditional methods typically require you to

- specify the intrinsic dimension,
- choose kernel parameters, or
- make structural assumptions about the manifold

Neural networks automatically adapt to intrinsic structure without requiring this prior knowledge"

Really?

Alternative story

- Can understand without advanced math background
- Scales well to high dimensions
- Function evaluations are highly parallelizable
- Flexible – universal approximators, can inject inductive bias
- Smooth recursive structure suited to calculating gradients

On top of — because of — this:

- Has received massive investment from the CS community
  - algos
  - software
  - hardware
- Many incremental improvements to improve regularization
- Steady increase in injection of domain-specific knowledge
  - CNNs
  - RNNs
  - transformers, etc.

DL with JAX

"JAX is a Python library designed for high-performance numerical computing, especially machine learning research."

"Its API for numerical functions is based on NumPy."

"Both Python and NumPy are widely used and familiar, making JAX simple, flexible, and easy to adopt."

JAX is more general-purpose than PyTorch or TensorFlow

- PT and TF both explicitly designed for deep learning
- JAX is a platform for scientific computing designed with machine learning / AI applications in mind

DL in JAX on one slide:

```python
def f(θ, x, σ=jnp.tanh):
    for W, b in θ:
        x = σ(x @ W + b)
    return x

def loss(θ, x, y):
  return jnp.sum((y - f(θ, x))**2)

loss_gradient = jit(grad(loss))

def train(θ, x_data, y_data, λ=0.01, m=1_000):
    for i in range(m):
        θ = θ - λ * loss_gradient(θ, x_data, y_data)
    return θ
```

JAX pytrees

Actually we cheated a little bit:

The call `θ = θ - λ * loss_gradient(θ, x_data, y_data)` doesn't actually work

Reason: We are attempting arithmetic operations on lists of tuples of arrays, not arrays

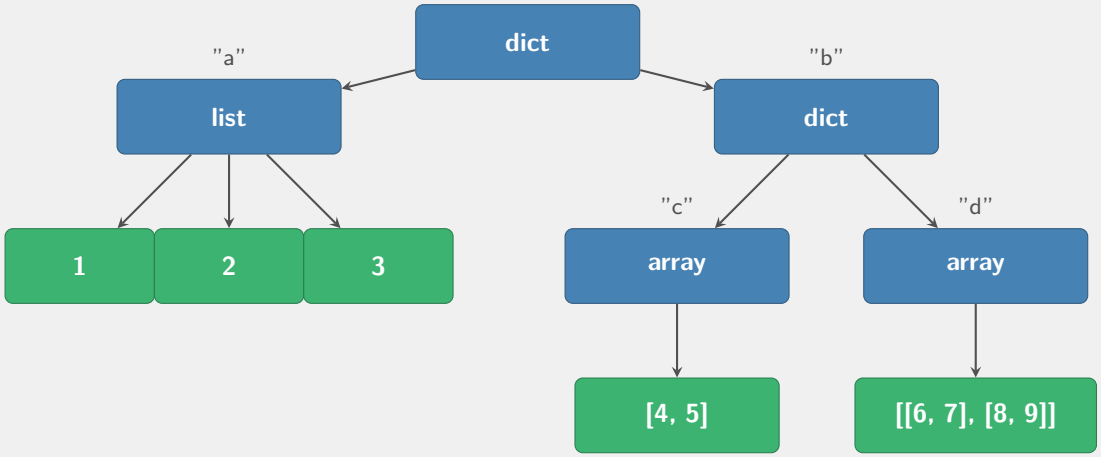
Because of this we need to modify the code a bit

We'll do this by using **pytrees** — which can work in many related settings

- a list of dictionaries
- a list of a dictionary of classes
- etc.

JAX PyTree Structure

```
pytree = {
    "a": [1, 2, 3],
    "b": {"c": jnp.array([4, 5]), "d": jnp.array([[6, 7], [8, 9]])}
}
```



Container nodes (dict, list, tuple)

Leaf nodes (arrays, scalars)

JAX can

- apply functions to all leaves in a pytrees structure
- differentiate functions with respect to the leaves of pytrees
- etc.

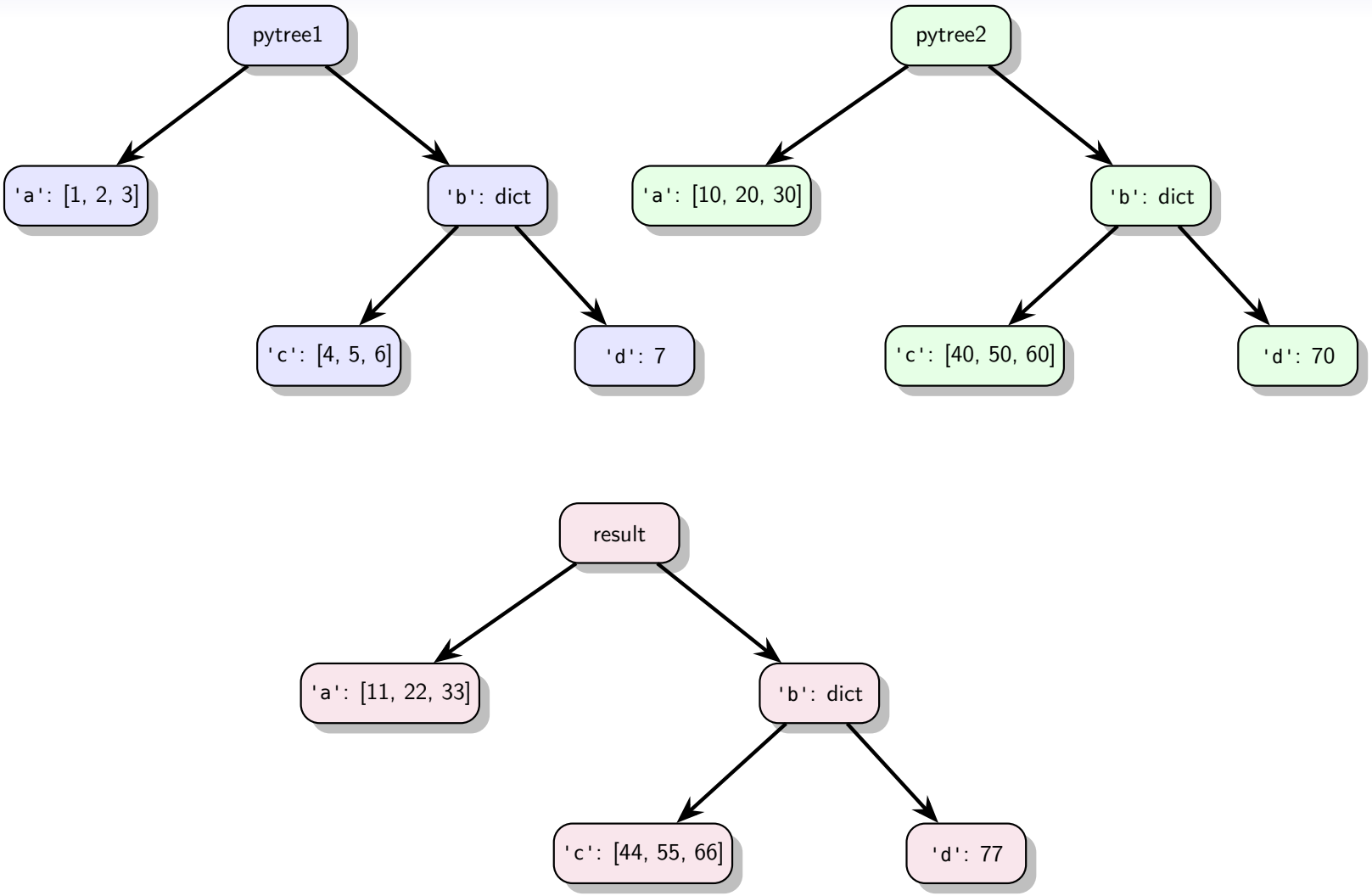


Figure: `jax.tree.map(lambda x, y: x + y, pytree1, pytree2)`

```python
def loss_fn(θ, x, y):
    pass  # Details omitted

def sgd_rule(p, g, λ=0.01):
    return p - λ * g  # Update rule for vectors

# Differentiate the loss function with respect to θ
loss_grad = jit(grad(loss_fn))

# Calculate pytree of gradients at θ
grads = loss_grad(θ, x, y)

# Update all parameters at once
new_θ = jax.tree.map(sgd_rule, θ, grads)
```